



CodeSmile

Master Test Plan

RIFERIMENTO	MasterTestPlan
VERSIONE	0.6
DESTINATARIO	Prof. Andrea De Lucia Dott. Giammaria Giordano



Laurea Magistrale in Software Engineering & IT Management –Università di Salerno
Corso di Ingegneria del Software: tecniche avanzate
Prof. Andrea De Lucia, Dott. Giammaria Giordano

Gestione Gruppo

MEMBRO	EMAIL	MATRICOLA
MARTINA DE LUCIA	m.delucia18@studenti.unisa.it	NF22500053
ALESSANDRA RAIA	a.raia7@studenti.unisa.it	NF22500054
CARLA STEFANILE	c.stefanile1@studenti.unisa.it	NF22500097



Version

DATA	VERSIONE	DESCRIZIONE	AUTORI
12/02/2026	0.1	Prima Stesura	MDL, AR, CS
12/02/2026	0.2	Seconda Stesura	MDL
12/02/2026	0.3	Terza Stesura	AR
14/02/2026	0.4	Quarta Stesura	CS
16/02/2026	0.5	Regression Testing, Scope del Testing	MDL
16/02/2026	0.6	Prima revisione	MDL



Sommario

Gestione Gruppo	2
Version	3
1 Introduzione	5
2 Obiettivi del Testing.....	5
3 Scope del Testing.....	6
3.1 Componenti in Scope	6
3.2 Componenti fuori Scope	6
4 Regression Testing	7
5 Strategia di Testing.....	8
5.1 Unit Testing (white-box).....	8
5.1.1 Prompt Engineering	8
5.1.2 Gui LLM Detection	9
5.1.3 Code Smell Management.....	9
5.2 Integration Testing (gray-box)	10
5.3 System Testing (black-box)	11
6 Coverage	12
7 Criteri Passed/Failed	12
8 Sospensione e ripristino	13



1 Introduzione

Il presente Master Test Plan definisce la strategia di testing per CodeSmile con riferimento alle modifiche introdotte da **CR01 (GUI mode)**, che aggiunge un'estensione **LLM opzionale** alla pipeline esistente. La CR01 introduce:

- controlli nella GUI principale per abilitare/disabilitare LLM, scegliere il provider (locale/API) e selezionare uno o più smell;
- una GUI dedicata al **Prompt Engineering** (gestione prompt per smell con versione *draft* e *default*, e “test su folder”);
- una GUI dedicata alla **gestione degli smell** (aggiunta/rimozione/modifica);
- una persistenza strutturata per catalogo smell/prompt/provider;
- tracciabilità dei risultati LLM nel report.

Vincolo fondamentale: **quando LLM è disattivo, il comportamento della pipeline standard deve rimanere invariato.**

2 Obiettivi del Testing

Gli obiettivi principali dell'attività di testing per CR01 sono:

1. **Regression/Baseline:** dimostrare che in modalità **LLM OFF** il sistema mantiene lo stesso comportamento e gli stessi output attesi della versione precedente.
2. **Conformità funzionale:** verificare che le funzionalità CR01 siano implementate correttamente:
 - attivazione LLM, selezione provider e smell (GUI Run);
 - gestione prompt per smell (draft/default) e persistenza;
 - gestione smell (CRUD) e propagazione nelle GUI;
 - “test su folder” tramite Prompt Engineering.
3. **Robustezza:** verificare la gestione di input non validi e condizioni d'errore (provider non selezionato/non raggiungibile, prompt vuoto, output LLM non normalizzabile, path non validi).
4. **Testabilità e ripetibilità:** garantire test automatici deterministici tramite provider mock e ridurre dipendenze ambientali.
5. **Tracciabilità nel report:** verificare che i risultati LLM siano distinguibili rispetto ai risultati standard (senza introdurre incompatibilità nell'output).



3 Scope del Testing

Il presente capitolo definisce il perimetro dell'attività di testing post-modification, identificando con precisione i componenti software oggetto di verifica e quelli esplicitamente esclusi, al fine di garantire una corretta allocazione delle risorse di testing e la tracciabilità rispetto agli obiettivi definiti al [Capitolo 2](#).

3.1 Componenti in Scope

Sono oggetto di testing i seguenti componenti software, introdotti o modificati dalla CR01:

Interfacce grafiche utente:

- `code_smell_detector_gui.py` – GUI principale, estesa con controlli per abilitazione LLM, selezione provider (Local/API) e selezione multipla smell;
- `prompt_engineering_gui.py` – GUI dedicata al Prompt Engineering (gestione prompt draft/default, test su folder);
- `manage_code_smells_gui.py` – GUI per gestione catalogo smell (operazioni CRUD).

Package `llm_detection` (architettura detection basata su LLM):

- `catalog_service.py` – Service layer per operazioni su catalogo smell/prompt/provider;
- `catalog_store.py` – Persistence layer per catalogo (serializzazione/deserializzazione JSON);
- `orchestrator.py` – Orchestrazione flusso detection (costruzione prompt, invocazione provider, normalizzazione output LLM);
- `providers.py` – Implementazione provider LLM (Local, API, Mock per testing);
- `types.py` – Definizioni type-safe per strutture dati (smell, prompt, provider, finding).

3.2 Componenti fuori Scope

Sono esplicitamente esclusi dal presente piano di testing i seguenti componenti:

Moduli non modificati dalla CR01:

- `cli/*` – Command Line Interface (invariata);
- `code_extractor/*` – Moduli di estrazione codice (dataframe, librerie, modelli, variabili);
- `data_preparation/*` – Pipeline di preparazione dataset (balanced dataset builder, code smell injector, dataset evaluator, repository downloader);
- `finetuning/*` – Train e validation modelli fine-tuned;
- `detection_rules/*` – Regole di detection statica;
- `webapp/*` – Interfaccia web;
- `report/*` – Moduli di generazione report (eccetto verifica tracciabilità risultati LLM).



4 Regression Testing

Obiettivo del regression testing è verificare che le modifiche introdotte da **CR01** non abbiano alterato il comportamento delle funzionalità pre-esistenti e della pipeline standard, in particolare nello scenario **LLM-mode OFF** (nessuna invocazione LLM e output standard invariato).

Strategia adottata:

La regressione viene condotta rieseguendo un insieme selezionato di test **pre-modification**, scelti in base ai componenti effettivamente modificati (principalmente `gui/code_smell_detector_gui.py`) e ai punti di integrazione più critici:

1. **Unit regression (pre-esistenti)**: riesecuzione dei test unitari della GUI base (`test/unit_testing/gui/test_code_smell_detector_gui.py`), in quanto coprono i flussi originali della GUI e le validazioni potenzialmente impattate dall'estensione LLM.
2. **Integration regression (pre-esistenti)**: riesecuzione dei test di integrazione che attraversano la GUI e verificano che l'interfaccia GUI→Analyzer e il flusso end-to-end della detection statica rimangano invariati (`test/integration_testing/test_full_integration_with_gui.py` e `test/integration_testing/test_gui_to_analyzer.py`).
3. **System regression (GUI senza LLM)**: riesecuzione di **tutti** i test di sistema pre-modification presenti in `gui_system_spec/test_tc_1_all.py` che verificano la modalità **GUI senza LLM**, per garantire non-regressione black-box sui principali scenari utente.

Criteri di superamento:

Il regression testing è considerato superato se:

- tutti i test selezionati risultano **PASS**; oppure
- eventuali **FAIL** sono tracciati nel *Test Incident Report Post Modification* e classificati come **non bloccanti** con motivazione (es. problema ambientale o test flaky) e assenza di impatto sul vincolo fondamentale "LLM OFF invariato".



5 Strategia di Testing

La strategia adotta tre livelli principali: **Unit, Integration, System**. L'approccio privilegia:

- unit test per logica e validazioni (massima stabilità e ripetibilità);
- integration test per flussi tra GUI/core/persistenza/report;
- system test black-box per scenari utente e gestione errori.

L'esecuzione dei test e i risultati sono accuratamente documentati nel documento *Test Incident Report Post Modification*.

5.1 Unit Testing (white-box)

Obiettivo: validare in isolamento le unità introdotte/modificate da CR01, con attenzione a condizioni e rami decisionali, massimizzando ripetibilità e determinismo tramite mock e I/O controllato.

5.1.1 Prompt Engineering

L'attività di unit testing per la funzionalità **Prompt Engineering** (UC02) è focalizzata su:

1. validazione della logica del controller GUI, con particolare attenzione a:
 - rami decisionali e validazioni lato interfaccia,
 - gestione dello stato interno (running, dirty, cancel),
 - interazione con il servizio di catalogo,
 - gestione degli effetti collaterali controllati (persistenza draft, generazione output),con l'obiettivo di garantire che:
 - le validazioni pre-run siano coerenti con i requisiti di UC02,
 - il comportamento sia corretto,
 - i messaggi di errore e le conferme utente siano coerenti con le specifiche,
 - non vengano introdotte regressioni sui flussi decisionali principali,
2. validazione dell'architettura che coinvolge gli LLM, con particolare attenzione a:
 - gestione catalogo e persistenza JSON,
 - validazioni smell e prompt,
 - provider mock/local/API (senza chiamate reali),
 - parsing e normalizzazione delle risposte LLM

Moduli testati

- `prompt_engineering/prompt_engineering_gui.py`
- `llm_detection/types.py`



- `llm_detection/catalog_store.py`
- `llm_detection/catalog_service.py`
- `llm_detection/providers.py`
- `llm_detection/orchestrator.py`

5.1.2 Gui LLM Detection

L'attività di unit testing per l'integrazione LLM nella GUI principale

(`gui/code_smell_detector_gui.py`) è condotta con strategia **white-box**, partendo da un'analisi dei branch decisionali introdotti dalla CR01. L'obiettivo è garantire che ogni costrutto condizionale rilevante (if/else e gestione eccezioni) sia eseguito in entrambi i rami, in modo da ridurre il rischio di bug nascosti nelle condizioni introdotte dalla manutenzione.

Focus del testing

1. **Validazione e flussi decisionali nella GUI**, con attenzione a:

- show/hide dei controlli LLM (toggle e gestione widget Tkinter);
- popolamento lista provider (Local/API) e gestione casi "lista vuota";
- selezione multi-smell e gestione smell non rilevabili/disabilitati;
- validazioni di pre-run (path input/output, num_walkers, combinazioni opzioni e prerequisiti LLM: provider selezionato, smell selezionati, provider esistente nel catalogo);
- messaggistica coerente e assenza di crash nei casi limite.

2. **Esecuzione LLM in isolamento**, con l'obiettivo di verificare il corretto comportamento del flusso LLM senza dipendenze esterne, tramite mock/stub del provider e I/O controllato. La verifica include:

- corretta selezione del provider;
- corretta costruzione dei target di analisi a partire dall'input;
- gestione deterministica dei risultati e
- gestione robusta degli errori.

Moduli testati

- `gui/code_smell_detector_gui.py`

5.1.3 Code Smell Management

L'attività di unit testing per la funzionalità di Code Smell Management è condotta con un approccio white box mirato a garantire la massima copertura e affidabilità delle operazioni di modifica del catalogo, isolando completamente la logica GUI dalle dipendenze del framework grafico (Tkinter).



1. Validazione della Business Logic e Interazione Utente: L'obiettivo primario è validare la correttezza dei tre Use Case fondamentali (Add, Modify, Remove) in un ambiente controllato, con particolare attenzione a:

- **Isolamento totale da Tkinter:** Utilizzo di tecniche avanzate di mocking (incluso il bypass di `init` tramite `__new__`) per testare la logica dei dialoghi senza istanziare finestre reali, garantendo velocità di esecuzione e stabilità in ambienti CI/CD.
- **Copertura dei rami decisionali:** Verifica sistematica di tutti i flussi alternativi e di errore, inclusi:
 - Validazione degli input (campi vuoti, caratteri non validi).
 - Gestione delle conferme utente (annullamento operazioni distruttive).
 - Reazione agli errori del service layer (eccezioni di persistenza o validazione).
- **State Management della GUI:** Controllo rigoroso dello stato dei widget (abilitazione/disabilitazione pulsanti, campi readonly) in risposta alle azioni dell'utente e allo stato del catalogo.

2. Verifica dell'Integrità del Catalogo: I test verificheranno che le operazioni UI si traducano correttamente in modifiche persistenti e coerenti, focalizzandosi su:

- **Invarianza degli ID:** Garanzia che gli identificativi degli smell (slug) rimangano immutabili durante le operazioni di modifica.
- **Prevenzione duplicati:** Verifica che il tentativo di inserire smell duplicati venga correttamente intercettato e gestito.
- **Callback e Sincronizzazione:** Controllo che le modifiche al catalogo inneschino correttamente gli aggiornamenti della vista principale e le callback di notifica.

Moduli testati

- `gui/manage_code_smells_gui.py` (Classe `ManageCodeSmellsGUI` e `AddSmellDialog`)

5.2 Integration Testing (gray-box)

Obiettivo: verificare l'interazione tra componenti, in particolare i flussi che attraversano GUI, persistenza e orchestrazione.

L'integration testing adotta un approccio gray-box, combinando la verifica del comportamento esterno dei componenti con una conoscenza parziale della loro struttura e responsabilità interne. L'attenzione è rivolta ai punti di integrazione introdotti dalla Change Request CR01 (LLM-Based Detection Extension), in particolare ai moduli che implementano il flusso di detection basato su Large Language Models.



Obiettivi principali:

- Verificare la corretta integrazione tra il service layer (CatalogService) e il persistence layer (CatalogStore) per garantire l'integrità dei dati utente durante operazioni di lettura, scrittura e aggiornamento.
- Validare il flusso di detection orchestrato, dalla costruzione dei prompt fino alla normalizzazione dei risultati, assicurando la corretta comunicazione tra orchestrator e provider.
- Individuare difetti di integrazione non rilevabili a livello unitario, quali errori di coordinamento, passaggio dati non corretto o gestione incompleta delle condizioni di errore.

L'approccio gray-box consente di progettare i test di integrazione tenendo conto delle interfacce e delle responsabilità dei moduli coinvolti, senza richiedere l'esercitazione esaustiva dei rami decisionali interni, che rimane demandata all'unit testing. I test garantiscono isolamento tramite componenti mock e file temporanei, assicurando esecuzione deterministica e compatibilità con pipeline CI/CD.

5.3 System Testing (black-box)

Obiettivo: validare CR01 come comportamento osservabile end-to-end (scenari utente), inclusi percorsi alternativi ed errori.

Al fine di soddisfare l'obiettivo si è scelto di **progettare** il testing di sistema utilizzando la tecnica di **Category Partition**.

La progettazione dei casi di test è basata su:

- i flussi di utilizzo della GUI con LLM, prompt engineering e gestione code smell,
- i casi d'uso principali e alternativi del sistema,
- le modalità di selezione degli input e degli output,
- le condizioni di errore e i casi limite legati all'interazione utente-sistema.

La definizione delle categorie, delle scelte e dei vincoli è raccolta in un documento di *Test Case Specification Post Modification* separato, al fine di mantenere il presente Master Test Plan focalizzato sulla strategia di testing.



6 Coverage

La qualità dei test unitari viene misurata tramite metriche di coverage (coverage.py).

Sono fissati i seguenti obiettivi minimi:

- **Statement Coverage $\geq 80\%$**
- **Branch Coverage $\geq 75\%$**

Le soglie sono applicate ai moduli introdotti o modificati da CR01, con particolare riferimento a:

- prompt_engineering_gui.py
- package llm_detection/*
- gui/manage_code_smells_gui
- gui/code_smell_detector_gui

Il mancato raggiungimento delle soglie comporta revisione e ampliamento dei casi di test, salvo motivata eccezione (es. codice astratto o rami non significativi).

Le evidenze quantitative di coverage sono riportate nel *Test Incident Report Post Modification*.

7 Criteri Passed/Failed

Un test di sistema è considerato **superato (PASSED)** o **fallito (FAILED)** sulla base dei seguenti criteri:

- **PASSED** se il comportamento osservato è conforme all'oracolo definito nel *Test Case Specification Post Modification*.
- **FAILED** in caso di deviazione dall'output atteso, crash non gestito o violazione dei requisiti.

L'attività di testing post-modification è considerata **completata con successo** se:

- tutti i test di unità e di integrazione risultano superati;
- tutti i test unitari soddisfano le soglie di coverage definite;
- i test di sistema progettati tramite Category Partition non evidenziano difetti bloccanti;
- eventuali anomalie sono documentate nel Test Incident Report e non compromettono la regressione con LLM OFF.



8 Sospensione e ripristino

In questa sezione verranno specificati i criteri di sospensione del test e di ripristino.

Criteri di sospensione

Il testing non verrà sospeso fino alla sua terminazione, anche in caso di rilevazione di una failure. Il testing potrà essere momentaneamente sospeso nel caso venga restituito, al momento dell'esecuzione, un errore nella definizione di uno dei test stessi.

Criterio di ripristino

Il testing verrà ripreso dopo aver risolto i fault individuati